Consolidated technical study

Carry-Free Representation in Pascal Rows and Mersenne Bases: The N(p) Bound

James Augustus

ORCID: 0009-0002-7286-7730

27 October 2026

Mathematical results, software models and open design decisions

Contents

Carry-Free Representation in Pascal Rows and Mersenne Bases: The N(p) Bound

- Abstract
- Scope and reading order
- I. Pascal rows in Mersenne bases
  - 1. Setting
  - 2. Results
  - 3. Proofs
  - 4. Computations
  - 5. Remarks
  - 6. Code
  - References
- II. N(p): the last carry-free row
  - 1. Setting
  - Worked examples: what is N(p) useful for?
  - 2. Results
  - 3. Lemmas
  - 4. Proofs
  - 5. Computations
  - 6. Open question
  - 7. Remarks
  - References
- III. Software and validation scope

Abstract

This consolidated study combines the representation of Pascal rows in Mersenne bases with N(p), the last carry-free row for a fixed base. Existing mathematical results, computed examples and design proposals are presented within one reading structure. The initial result selects row n = p + 2. The N(p) extension determines how far rows may advance in the same base. Large parameters do not imply that every digit has been computed or stored.

Scope and reading order

Part I gives the representation, the n = p + 2 result and central-coefficient relations. Part II contains the definition of N(p), ten worked examples, the threshold sequence, bounds and proofs. Part III describes the software scope and recorded validation.

This edition is a technical study authored by James Augustus. The technical content and proofs of its source documents are retained. A common title and reading structure have been added, while repeated document titles, author lines and separate contents tables have been removed. Section numbers and numbered references are local to their respective main parts. The author’s narrative is retained in source sections describing the research history.

Code and tests are included in the accompanying repository archive. Text is CC BY 4.0 and original code is MIT licensed. Third-party dependencies retain their own licenses. Software checks alone do not establish physical hardware, speed or energy advantages.

I. Pascal rows in Mersenne bases

1. Setting

For any base b, (b + 1)^n = Σ_{k=0}^{n} C(n,k) b^(n−k). The coefficients are the base-b digits of (b + 1)^n if and only if every C(n,k) is less than b, that is, if the middle coefficient is less than b. In base 10 this holds for 11^n with n ≤ 4 and fails from n = 5 on.

We take b = M_p, so that b + 1 = 2^p, and n = p + 2. This is where the exponent comes from, (b + 1)^n = `(2^p)^(p+2)` = 2^(p·(p+2)). Here p is an integer with p ≥ 2. The carry-free result applies to every integer p ≥ 7. For the odd-p results in Theorem 3 and Corollary 4, write n = 2x + 1 and x = (p + 1)/2; the two middle coefficients are C(n, x) = C(n, x + 1).

p	Mersenne exponent	7
$M_p = 2^p - 1$	base	127
n = p + 2	row number	9
x = (p + 1)/2	position of the middle	4
k	position in the row, from 0	0 … 9
d	distance from the middle	0 … 4
C(n, k) = n!/(k!(n − k)!)	k-th entry of row n	C(9, 3) = 84
$Cat_x = C(2x, x)/(x + 1)$	Catalan number	$Cat_4 = 14$

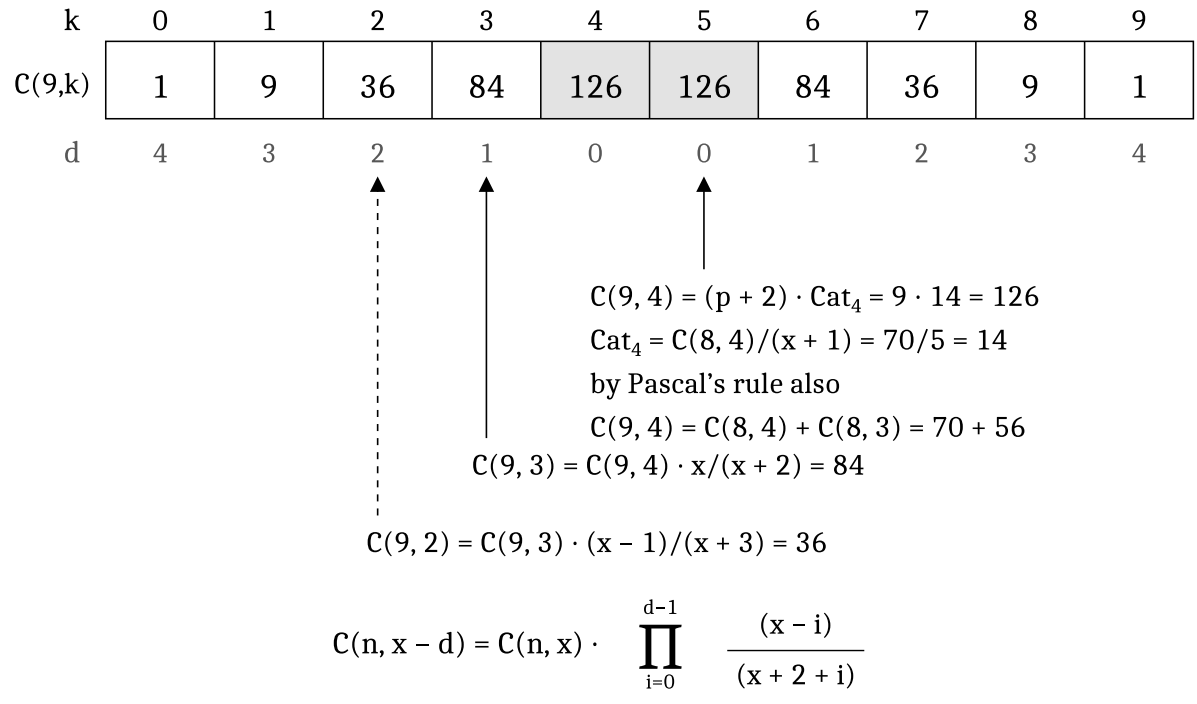


Figure 1. Row n = p + 2 = 9 in base $M_7 = 127$ (p = 7, x = 4).
k is the position, d the distance from the middle.
The row is symmetric, C(n, k) = C(n, n − k).

Figure 1: row coordinates

2. Results

Theorem 1. $\Sigma_{k=0}^{n} C(n,k) M_p^{n-k} = 2^{p\cdot(p+2)}$.

Theorem 2. For every integer $p \geq 7$, $C(p + 2, \lfloor (p + 2)/2 \rfloor) < M_p$. Hence every entry of row $p + 2$ is a base-M_p digit, and $2^{p\cdot(p+2)}$ is written in base M_p as that row.

Theorem 3. For odd $p \geq 7$, let $x = (p + 1)/2$. Then $C(2x + 1, x) = (2x + 1)\,\mathrm{Cat}_x$, where $\mathrm{Cat}_x = C(2x, x)/(x + 1)$. In particular the middle digit equals $(p + 2)\,\mathrm{Cat}_{(p+1)/2}$, and the second digit $p + 2$ divides it.

Corollary 4. For odd $p \geq 7$, every prime q with $(p + 3)/2 < q \leq p + 2$ divides the middle digit exactly once. If $(p + 3)/2$ is prime, it does not divide the middle digit.

Example, $p = 7$: base 127, $2^{63} = 128^9$ has digits 1, 9, 36, 84, 126, 126, 84, 36, 9, 1, and $126 = 9 \cdot 14 = 9 \cdot \mathrm{Cat}_4$. The prime $5 = (p + 3)/2$ does not divide 126.

3. Proofs

Theorem 1 is the binomial theorem with $b = M_p$.

Theorem 2. Induct separately over odd and even p, increasing p by 2. For odd p, start at $n = p + 2 = 9$: $C(9, 4) = 126 < 127 = M_7$. For the step,

$$C(2x + 3, x + 1) / C(2x + 1, x) = (2x + 3)(2x + 2) / ((x + 1)(x + 2)) = 2(2x + 3)/(x + 2) < 4.$$

If $C(2x + 1, x) < M_p$, then $C(2x + 3, x + 1) < 4\,M_p < 4\,M_p + 3 = M_{p+2}$. For even p, start at $p = 8$: $C(10, 5) = 252 < 255 = M_8$. Writing $n = 2x$, the next middle coefficient satisfies

$$C(2x + 2, x + 1) / C(2x, x) = 2(2x + 1)/(x + 1) < 4.$$

Thus the same comparison with $M_{p+2} = 4\,M_p + 3$ proves the even-p step. The two inductions cover all integers $p \geq 7$. For $p = 2, 3, 4, 5, 6$, the middle coefficients are respectively 6, 10, 20, 35, 70, exceeding the bases 3, 7, 15, 31, 63.

Theorem 3. $C(2x + 1, x) = (2x + 1)! / (x!\,(x + 1)!) = (2x + 1) \cdot (2x)! / (x!\,(x + 1)!) = (2x + 1)\,C(2x, x)/(x + 1)$.

Corollary 4. By Legendre's formula the exponent of q in $C(n, x)$ is $v_q(n!) - v_q(x!) - v_q((x + 1)!)$. If $x + 1 < q \leq n$, then $n < 2q$, so $v_q(n!) = 1$ and the other two terms vanish. If $q = x + 1$, then $v_q((x + 1)!) = 1$ and the exponent is 0.

4. Computations

The tables record middle-coefficient calculations for the 52 Mersenne prime exponents in the 14 October 2026 working list, $p = 2, \ldots, 136279841$. They were calculated using gmpy2 and checked by a separate script that directly recomputed the middle coefficients with gmpy2.comb; its archived output reports 52 rows and no mismatches. Both calculations use the GMP backend, so this is not verification by an independent arithmetic implementation. That historical check covers the table fields and middle coefficients, not a traversal or full-value reconstruction of each Pascal row by the CLI below. Selected rows:

p	bits of middle digit	bits of M_p	decimal digits of middle digit
7	7	7	3

p	bits of middle digit	bits of M_p	decimal digits of middle digit
13	13	13	4
31	31	31	10
127	126	127	38
521	519	521	156
1279	1276	1279	384
132049	132043	132049	39749
82589933	82589922	82589933	24862044
136279841	136279830	136279841	41024317

The gap in bits is about ½ $\log_2$ p − 2, as expected from C(2x, x) ~ 4^x / √(πx). For 7 ≤ p ≤ 37, row p + 2 is the last clean row in base M_p; from p = 38 on, row p + 3 is clean as well.

5. Remarks

1. Primality plays no role. The carry-free row result holds for every integer p ≥ 7; Theorem 3 and Corollary 4 retain the odd-p assumption; for example M_11 = 23 · 89 gives a clean row. The Mersenne prime exponents are used only as test cases.
2. The ingredients are standard: the binomial theorem, Legendre's formula, the identity of Theorem 3, and the extension of the powers-of-11 pattern to arbitrary bases [2, 3]. I have not found the choice b = M_p, n = p + 2 in the literature, but the search was brief.
3. The digits can be produced one from the next, R_(k+1) = R_k (n − k)/(k + 1), and the value recovered by Horner's rule. This recurrence multiplies a growing coefficient by an integer no larger than n and performs an exact division; these operations have a cost. Recovering the full value with Horner additionally requires large-integer multiplications.
4. The middle digit can also be found by addition alone, without the Catalan number: each middle is the previous middle plus its neighbour, C(9, 4) = C(8, 4) + C(8, 3) = 70 + 56. My first formula contained a Catalan number; I later rewrote it in this additive form.

6. Code

`mersenne_rows.py` and `mersenne_rows_gmp.c` (C with GMP) generate the coefficients and report whether each is below the base as `clean`. They print the row only for p ≤ 20. They compare the value with 2^(p·(p+2)) modulo those of three candidate primes that are coprime to the base; `moduli=m/3` reports the number used. Moduli sharing a factor with the base are skipped to avoid a vacuous check. If none remains, the programs report `value(mod)=unavailable`, `clean=unchecked` and exit with status 1 without calculating the row or full value. The `full` option also compares the exact integers.

The value checks test implementation consistency with the binomial identity; they are not an independent mathematical proof or a primality test. The substantive representation condition is that the largest coefficient is below the base. Both programs traverse n + 1 coefficients; suppressing row output does not remove that work. The `full` result has p(p + 2) + 1 bits. For very large exponents, this traversal and full-value storage may be impractical; directly computing the middle coefficient is a different task.

```
cc -O2 -o mersenne_rows_gmp mersenne_rows_gmp.c -lgmp
./mersenne_rows_gmp 7 full
./mersenne_rows_gmp 132049
```

The folder `tables/` holds four tables from the 52-exponent study: the theorems with their proofs, the middle digits, the middle digits of the first 50, and their factorizations. A GMP-free check for p = 7, …, 61 is test T21_mersenne_row in `journal_tests.c`, in the companion repository `ctcpu-hunter`.

References

1. Binomial theorem. https://mathworld.wolfram.com/BinomialTheorem.html
2. L. Jiu, C. Vignat, On binomial identities in arbitrary bases, J. Integer Sequences 19 (2016). https://arxiv.org/abs/1602.04149
3. M. S. Islam et al., Generating binomial coefficients in a row of Pascal’s triangle from extensions of powers of eleven, Heliyon (2022). https://arxiv.org/abs/2005.14490
4. Legendre’s formula. https://en.wikipedia.org/wiki/Legendre%27s_formula
5. OEIS A001405 (central binomial coefficients), A000108 (Catalan numbers). https://oeis.org
6. GIMPS, list of known Mersenne primes. https://www.mersenne.org/primes/

Text CC BY 4.0, code MIT.

II. N(p): the last carry-free row

1. Setting

Throughout, p is an integer with p ≥ 2, so M_p is a valid positional base. Primality of p or M_p is not required. Write c(n) = C(n, ⌊n/2⌋) for the largest entry of row n. For any base b, (b + 1)^n = Σ_{k=0}^{n} C(n,k) b^k, and the coefficients are the base-b digits of (b + 1)^n if and only if c(n) < b. I fix b = M_p and let the row vary:

N(p) = max{ n : c(n) < M_p }, D(p) = N(p) − p, h(p) = ½ $\log_2(\pi p/2)$.

Lemma 1 below shows that the clean rows form an initial segment {0, 1, …, N(p)}, so N(p) is found by stopping at the first row that fails. I also use

w(m) = 4^m / C(2m, m) (m ≥ 1), m_k = min{ m : 2^k C(2m, m) < 4^m }, p_k = 2m_k − k − 1.

Worked examples: what is N(p) useful for?

N(p) gives the validity boundary when Pascal coefficients are used as the digits of a number. With the base fixed, it tells us how far we can go through the rows. In the reverse direction, it helps select a sufficient base for a desired row. The examples below illustrate these uses. They are not new speed or compression results.

Example 1. A number small enough to check by hand

For p = 3 the base is 7. Pascal row 4 is [1, 4, 6, 4, 1]. Its largest coefficient is 6 < 7, so all entries are valid digits. Indeed,

$1 \times 7^4 + 4 \times 7^3 + 6 \times 7^2 + 4 \times 7 + 1 = 4096 = 8^4 = 2^{12}$.

Row 5 is [1, 5, 10, 10, 5, 1]. The value 10 is not a base-7 digit. The coefficient expansion still equals 8^5 = 32768, but the actual digits are [1, 6, 4, 3, 5, 1]. The right-hand 10 gives remainder 3 and carry 1. The next 10 becomes 11 after that carry, giving remainder 4 and carry 1. Its neighboring 5 becomes 6. Thus N(3) = 4. This p = 3 example illustrates the definition; it is outside Theorem A, which requires p ≥ 7.

Example 2. Equality at the boundary also fails

For p = 2 the base is 3. Row 2, [1, 2, 1], fits. Row 3, [1, 3, 3, 1], does not, because the only valid digits are 0, 1 and 2. Hence N(2) = 2. The check must be c(n) < b, not c(n) ≤ b. A program accepting equality would produce an invalid digit sequence.

Example 3. Seeing the complete row in base 127

For p = 7 the base is 127. Row 9 is [1, 9, 36, 84, 126, 126, 84, 36, 9, 1] and represents 2^{63}. Its central coefficient is 126 < 127; the next row has central coefficient 252. Therefore N(7) = 9. In this example the first note’s choice p + 2 reaches the boundary exactly.

Example 4. Going beyond the guaranteed row

For p = 38 the situation is different:

Row	Central coefficient	Comparison with base 274877906943
40	137846528820	Smaller, coefficients remain digits

Row	Central coefficient	Comparison with base 274877906943
41	269128937220	Smaller, coefficients remain digits
42	538257874440	Larger, carrying is required

N(38) = 41. Thus p + 2 = 40 is a safe choice, but not the last valid choice. A converter can accept row 41 in this base; it cannot accept the coefficients of row 42 directly as digits.

Example 5. Choosing a base for a desired row

Suppose we want to write row 42 without carrying. Its central coefficient is 538257874440. The p = 38 base is insufficient. For p = 39 the base is 549755813887, larger than the central coefficient. Therefore 39 is the smallest sufficient p among Mersenne bases. The next central coefficient, 1052049481860, also confirms N(39) = 42.

The general decision rule is to choose the smallest p ≥ 2 satisfying c(n) < 2^p − 1 for the desired row n. This reverses the question of finding N(p) for a fixed p. It is sufficient to check the largest coefficient rather than every entry separately.

Example 6. A number longer than one million bits

For p = 1000 the base is b = $2^{1000} - 1$ and the last row is N(1000) = 1005. The beginning and end of that row are

[1, 1005, 504510, …, 504510, 1005, 1].

There are 1006 digits in total. The … represents 1000 omitted Pascal coefficients, not zeros. The largest coefficient has 301 decimal digits. All coefficients are below the base. With their positional weights, these digits represent $2^{1\,005\,000}$.

In binary, this number is a single 1 followed by 1 005 000 zeros. Its exact binary length is therefore 1 005 001 bits. The use here is to know precisely which Pascal row represents a large number without carrying. There is no need to print its entire decimal expansion.

Using 1006 digits does not by itself compress data. Each digit may require up to 1000 bits. The number of digits and the number of bits required to store them are different quantities.

Example 7. A number whose raw binary form needs about 125 GB

Choose p = 1 000 000. Since the thresholds give 667533 ≤ p < 2670166, we have N(p) − p = 10 and N(p) = 1 000 010. The base is b = $2^{1\,000\,000} - 1$.

[1, 1000010, 500009500045, …, 500009500045, 1000010, 1].

The row contains 1 000 011 digits. The omitted middle contains 1 000 005 coefficients. The represented number is $2^{1\,000\,010\,000\,000}$, with exact binary length 1 000 010 000 001 bits.

Storing those bits without compression in eight-bit bytes requires 125 001 250 001 bytes, about 125 GB. By contrast, “2 raised to 1 000 010 000 000” is a short symbolic description. A symbolic description and expanding every bit in memory are different operations. This example does not generate the entire huge number or row. The last row is determined from the mathematical threshold result.

Example 8. A parameter of one trillion

Let $p = 10^{12} = 1\ 000\ 000\ 000\ 000$. At this scale we can use the proved error band instead of trying every row. For $h(p) = \frac{1}{2}\log_2(\pi p/2)$, we have $20.257 < h(p) < 20.258$. Combining this with $-1 < E(p) \le 0.270575$ from the mathematical note gives

$$19.257 < N(p) - p < 20.529.$$

$N(p) - p$ is an integer. The only integer in this interval is 20. Therefore $N(p) = 10^{12} + 20 = 1\ 000\ 000\ 000\ 020$.

Writing $n = 10^{12} + 20$, the row has the form $[1, n, n(n-1)/2, \ldots, n(n-1)/2, n, 1]$. There are $10^{12} + 21$ digits. The represented number is $2^{10^{24} + 20 \times 10^{12}}$. This does not mean every coefficient was computed individually. The last row was determined by selecting the unique integer permitted by the proved bound.

Example 9. When p itself has 101 decimal digits

Now let $p = 10^{100}$. In decimal, this is a single 1 followed by 100 zeros. The base is $b = 2^{10^{100}} - 1$. We have $166.422 < h(p) < 166.423$. The same error band gives

$$165.422 < N(p) - p < 166.694.$$

The only possible integer is 166. Therefore $N(10^{100}) = 10^{100} + 166$ is determined exactly.

With $n = 10^{100} + 166$, this enormous row again has the form

$$[1, n, n(n-1)/2, \ldots, n(n-1)/2, n, 1].$$

Its exact digit count is $10^{100} + 167$. It represents $2^{10^{200} + 166 \times 10^{100}}$. The 1 at each end is an endpoint of the Pascal row. The … stands for specific omitted binomial coefficients of row n, not unknown or arbitrary values.

The use of $N(p)$ is especially clear here: mathematics determines the carry-free boundary for a family of rows that cannot be printed in full. This does not claim that we stored the complete row or computed and displayed every entry.

Example 10. Boundary pairs for checking a program

p	N(p)	First row that is not carry-free
2	2	3
3	4	5
4	5	6
7	9	10
8	10	11
37	39	40
38	41	42
39	42	43
100	103	104
158	161	162

p	N(p)	First row that is not carry-free
159	163	164
1000	1005	1006

Each table entry supplies two checks: row N(p) must fit, while row N(p) + 1 must not. These pairs can be used to check boundary handling in a base converter or a program that uses Pascal coefficients as digits. Changing p also changes the base. Jumps in this table do not mean that a row is skipped within one fixed base.

Scope of the uses

N(p) is a mathematical tool defining the exact valid range of this representation. It can illustrate carrying in teaching, guide base selection in an algorithm, and supply boundary checks. By itself, it does not show that generating a complete Pascal row or a very large power is faster. It does not provide a separate encryption method, a general compression result, or a primality test. The cost of computing N(p) also depends on the method used.

2. Results

Theorem A. For every integer $p \geq 7$, $N(p) \geq p + 2$.

This is Theorem 2 of the note on powers of two as carry-free Pascal rows [1].

Theorem B1 (characterization). For every $p \geq 7$ and every $k \geq 2$, $N(p) - p \geq k$ if and only if $p \geq p_k$. Hence $N(p) - p = \max\{ k : p_k \leq p \}$. It never decreases in p and increases by exactly 1 at each p_k.

k	m_k	p_k
2	5	7
3	21	38
4	82	159
5	326	646
6	1304	2601
7	5215	10422
8	20861	41713
9	83443	166876
10	333772	667533
11	1335089	2670166
12	5340354	10680695

So $N(p) - p$ is 2 for $7 \leq p \leq 37$, 3 for $38 \leq p \leq 158$, 4 for $159 \leq p \leq 645$, 5 for $646 \leq p \leq 2600$, 6 for $2601 \leq p \leq 10421$, and 7 from $p = 10422$ to $p = 41712$.

Theorem B2 (thresholds). For every $k \geq 2$,

$4^k/\pi - 1/4 - 0.07/m_k < m_k < 4^k/\pi + 3/4 + 1/(36(m_k - 1))$.

Hence $p_k = (2/\pi)4^k - k - 3/2 + 2\varphi_k$ with $-0.07/m_k < \varphi_k < 1 + 1/(36(m_k - 1))$, and in particular

$2\times4^k/\pi - k - 1.53 < p_k < 2\times4^k/\pi$.

For $1 \le k \le 12$, $m_k = \lceil 4^k/\pi - 1/4 \rceil$, so $p_k = 2\lceil 4^k/\pi - 1/4 \rceil - k - 1$ and, for $7 \le p \le 42\,700\,000$, $N(p) - p = \max\{ k \le 12 : 2\lceil 4^k/\pi - 1/4 \rceil - k - 1 \le p \}$.

Theorem B3 (error band). For every $p \ge 7$,

$h(p) - 1 < N(p) - p \le h(p) + E_{max}$,

$E_{max} = 2 - h(7) = 0.270574474235\ldots < 0.270575$.

The exact upper value E_{max} is attained only at $p = 7$. The decimal 0.270575 is an outward rounding, not an attained endpoint. The lower value -1 is never attained but is approached arbitrarily closely. If $N(p) - p = k \ge 3$, the upper bound improves to

$N(p) - p - h(p) < e(k) = -\tfrac{1}{2}\log_2(1 - (\pi/2)(k + 1.53)/4^k)$,

with $e(3) < 0.0851$, $e(4) < 0.0250$, $e(5) < 0.0073$, $e(6) < 0.0021$, $e(7) < 0.00060$. In integer form, $N(p) - p = \lfloor h(p) \rfloor + 1$ if $p_k \le p < 2\times4^k/\pi$ for some k, and $N(p) - p = \lfloor h(p) \rfloor$ otherwise. Each such window contains between k and $k + 2$ integers: 7-10, 38-40, 159-162, 646-651, 2601-2607, 10422-10430.

Theorem B4 (ratios). For every $k \ge 2$, $3k - 5 < p_{(k+1)} - 4p_k < 3k + 6$. Hence $p_{(k+1)}/p_k > 4$ and $p_{(k+1)}/p_k \to 4$, with $p_{(k+1)}/p_k - 4 \approx 3\pi k/(2\times4^k)$.

p_k	$p_{(k+1)}$	$p_{(k+1)} - 4p_k$	$p_{(k+1)}/p_k$
38	159	7	4.184210
159	646	10	4.062893
646	2601	17	4.026315
2601	10422	18	4.006920
10422	41713	25	4.002398
41713	166876	24	4.000575
166876	667533	29	4.000173
667533	2670166	34	4.000050
2670166	10680695	31	4.000011

Theorem C (threshold rows). Let $k \ge 3$, $p = p_k$ and $N = N(p)$. Then $N = 2m_k - 1$ is odd and

$1 - 1/(N + 1) \le c(N)/2^p < c(N)/M_p < 1$.

For example $c(N)/M_p$ is 0.979085 at $p = 38$ ($N = 41$), 0.995350 at $p = 159$, 0.999538 at $p = 646$, 0.999826 at $p = 2601$ and 0.999994 at $p = 10422$.

3. Lemmas

Lemma 1 (monotonicity). $c(0) = c(1) = 1$, and $c(n + 1) > c(n)$ for every $n \ge 1$. Moreover $c(n + 1) \le 2c(n)$ for all n, with $c(n + 1) = 2c(n)$ exactly when n is odd.

Proof. For $n = 2x$, $c(2x + 1)/c(2x) = (2x + 1)/(x + 1)$, which lies in $[1, 2)$ and equals 1 only at $x = 0$. For $n = 2x + 1$, Pascal's rule and symmetry give $c(2x + 2) = C(2x + 1, x) + C(2x + 1, x + 1) = 2c(2x + 1)$.

Thus c(n + 1) = c(n) holds only at n = 0, whereas c(n + 1) = 2c(n) holds exactly for odd n.

Lemma 2. c(n) = 2^q − 1 only for n = 0, 1 (c = 1) and n = 3 (c = 3). Consequently, for n ≥ 4, c(n) < 2^p − 1 if and only if c(n) < 2^p.

Proof. For even n ≥ 2, c(n) = 2c(n − 1) is even. For n = 2x + 1, c(n) = C(2x + 2, x + 1)/2, and Legendre's formula gives $v_2(C(2y, y)) = s_2(y)$, the number of ones in the binary expansion of y [3]. Indeed, $v_2(t!) = \Sigma_{j\geq 1} \lfloor t/2^j \rfloor = t - s_2(t)$, and $s_2(2y) = s_2(y)$. So c(n) is odd exactly when x + 1 is a power of two, that is, n = 2^M − 1. For M ≥ 3 write

C(2^M − 1, 2^(M−1) − 1) = ∏_{i=1}^{K} (2^M − i)/i, K = 2^(M−1) − 1.

With i = 2^v u, u odd, the factor is (2^(M−v) − u)/u, a ratio of odd integers congruent to −1 modulo 2^(M−v). Since i < 2^(M−1), v ≤ M − 2. Factors with v ≤ M − 3 are ≡ −1 (mod 8). The single factor with v = M − 2 is i = 2^(M−2), equal to 3. As K is odd, the product is ≡ (−1)^(K−1) × 3 = 3 (mod 8), whereas 2^q − 1 ≡ 7 (mod 8) for q ≥ 3. Since c(7) = 35, the case M ≥ 3 would need q ≥ 6. The remaining cases M ≤ 2 are n = 0, 1, 3.

Lemma 3 (parity). For every n ≥ 1, 2^n/c(n) = w(⌈n/2⌉). Further w(m + 1)/w(m) = 2(m + 1)/(2m + 1) > 1, so w is strictly increasing.

Proof. For n = 2m this is the definition. For n = 2m − 1, C(2m, m) = 2C(2m − 1, m − 1) by Lemma 1, so 2^(2m−1)/C(2m − 1, m − 1) = 4^m/C(2m, m). The ratio follows from C(2m + 2, m + 1) = C(2m, m)(2m + 1)(2m + 2)/(m + 1)².

Thus 2^n/c(n) grows only from an even row to the next odd row and is unchanged from an odd row to the next even row.

Lemma W. For every m ≥ 1,

π(m + 1/4 − 1/(36m + 3)) < w(m)² < π(m + 1/4 + 0.07/m).

Proof. Robbins' form of Stirling's formula [2] states n! = √(2πn)(n/e)^n e^(r_n) with 1/(12n + 1) < r_n < 1/(12n). Applied to (2m)! and m!, it gives w(m)² = πm e^s with s = 2(2r_m − r_(2m)), and

ms > (36m − 1)/(12(12m + 1)) = 1/4 − 1/(36m + 3), ms < (18m + 1)/(72m + 3) = 1/4 + 1/(288m + 12).

Now e^s > 1 + s, and for 0 < s < 1, e^s < 1 + s + s², since the tail of the exponential series is at most s²(e − 2). Here s < 19/(75m), so ms² < 0.0642/m, and 1/(288m + 12) + 0.0642/m < 0.07/m.

By rows, with θ the error term: for even n, (2^n/c(n))² = (π/2)(n + 1/2 + θ) with −2/(18n + 3) < θ < 0.28/n. For odd n, (2^n/c(n))² = (π/2)(n + 3/2 + θ) with −2/(18n + 21) < θ < 0.28/(n + 1). The two parities differ by about 0.72/n in $\log_2 c(n)$, which is small. It decides which row is the threshold row (Theorem C).

4. Proofs

Theorem A. For every n, c(n + 2)/c(n) equals 2(2x + 1)/(x + 1) = 4 − 2/(x + 1) when n = 2x and 2(2x + 3)/(x + 2) = 4 − 2/(x + 2) when n = 2x + 1, so c(n + 2) < 4c(n). The base cases are C(9, 4) = 126 < 127 = M_7 and C(10, 5) = 252 < 255 = M_8. If c(p + 2) ≤ M_p − 1, then c(p + 4) < 4(2^p − 2) = 2^(p+2) − 8 < M_(p+2). The odd chain starts at p = 7 and the even chain at p = 8. Theorem A also follows from Theorem B1, since p_2 = 7.

Theorem B1. Let p ≥ 7 and k ≥ 2, so p + k ≥ 9. Then

$N(p) - p \geq k \iff c(p+k) < 2^p - 1$ (Lemma 1) $\iff c(p+k) < 2^p$ (Lemma 2) $\iff w(\lceil (p+k)/2 \rceil) > 2^k$ (Lemma 3) $\iff \lceil (p+k)/2 \rceil \geq m_k \iff p \geq 2m_k - k - 1$.

By Lemma W, w is unbounded, so m_k exists. The ratio $w(m+1)/w(m)$ is less than 2. Since $w(m_k - 1) \leq 2^k$, this gives $w(m_k) < 2^{k+1}$, hence $m_{k+1} \geq m_k + 1$ and $p_{k+1} > p_k$.

Theorem B2. From $w(m_k)^2 > 4^k$ and the upper bound of Lemma W, $4^k/\pi < m_k + 1/4 + 0.07/m_k$. From $w(m_k - 1)^2 \leq 4^k$ and the lower bound, $m_k - 3/4 - 1/(36m_k - 33) < 4^k/\pi$. Substitute into $p_k = 2m_k - k - 1$ and use $m_k \geq 5$ for $k \geq 2$. The resulting wider interval always contains $\lceil 4^k/\pi - 1/4 \rceil$, and contains a second integer only if the fractional part of $4^k/\pi - 1/4$ is below $0.07/m_k$ or above $1 - 1/(36(m_k - 1))$. The values for $k \leq 12$ were computed exactly (Section 5). Since $p_{13} > 2 \times 4^{13}/\pi - 14.53 > 42\,722\,815$, the closed form for $N(p)$ holds up to 42 700 000.

Theorem B3. Let $N(p) - p = k$, so $p_k \leq p < p_{k+1}$. Lower bound: $p < p_{k+1} < 2 \times 4^{k+1}/\pi$, so $\pi p/2 < 4^{k+1}$ and $h(p) < k + 1$. Upper bound: $p > 2 \times 4^k/\pi - k - 1.53$, so $\pi p/2 > 4^k(1 - (\pi/2)(k + 1.53)/4^k)$, and taking logarithms gives $e(k)$. On the plateau $k = 2$, $p \geq 7$ gives the value $2 - h(7)$. Since $e(k)$ decreases and $e(3) < 0.0851 < 2 - h(7)$, $2 - h(7)$ is the overall maximum. Within a plateau h increases, so the extreme values sit at $p = p_k$ and $p = p_{k+1} - 1$, and the latter tends to -1 by Theorem B2. For the integer form, $h(p)$ lies in $(k - e(k), k + 1)$ on the plateau, and $h(p) \geq k$ exactly when $p \geq 2 \times 4^k/\pi$.

For the window count, write $T_k = 2 \times 4^k/\pi$. It contains $\lceil T_k \rceil - p_k = k + 1 + \lceil 1/2 - 2\varphi_k \rceil$ integers. The bounds in B2 give $-1.514 < 1/2 - 2\varphi_k < 0.528$ for $k \geq 2$, so the last ceiling is -1, 0 or 1.

Theorem B4. Writing Theorem B2 for k and k + 1 gives $p_{k+1} - 4p_k = 3k + 7/2 + 2\varphi_{k+1} - 8\varphi_k$. Since $m_2 = 5$ and $m_3 = 21$, this lies above $3k + 3.5 - 0.03 - 8.06 > 3k - 5$ and below $3k + 3.5 + 2.01 + 0.12 < 3k + 6$. For $k \geq 2$, $3k - 5 > 0$, and $(3k + 6)/p_k \to 0$ because $p_k > 2 \times 4^k/\pi - k - 1.53$.

Theorem C. At $p = p_k$, $N(p) = p + k$ and $N(p - 1) = N - 2$, so $c(N - 1) \geq 2^{(p-1)} - 1$ and, by Lemma 2, $c(N - 1) \geq 2^{(p-1)}$. If N were even, $c(N) = 2c(N - 1) \geq 2^p > M_p$, a contradiction. For $N = 2x + 1$, $c(N) = c(2x)(2x + 1)/(x + 1) \geq 2^{(p-1)}(2 - 1/(x + 1)) = 2^p(1 - 1/(N + 1))$. Directly from Theorem B1, $N = p_k + k = 2m_k - 1$.

5. Computations

The accompanying verifier checks finite instances of the statements. From the repository root, Python 3.10 or later is sufficient:

```
python3 -B tests/verify_np.py --quick
python3 -B tests/verify_np.py
```

The default run checks every p from 7 to 20000. The quick run stops at p = 1000 and is a smoke test, not a substitute for the default run. Neither mode writes a file unless `--json PATH` is supplied.

Check	Default run	Quick run
Central coefficients from recurrence and math.comb	n = 0..20012	n = 0..1012
N(p), Theorem A, error bounds and threshold characterization	p = 7..20000	p = 7..1000
Pascal addition-only oracle for N(p)	p = 7..128	p = 7..128
Lemma W	m = 1..10006	m = 1..506

Check	Default run	Quick run
Direct scan for m_k	k = 1..8	k = 1..8
Exact threshold boundary checks	k = 1..12	k = 1..8
Special residue c(2^M − 1) modulo 8	M = 3..20	M = 3..12
Ratio bounds	k = 2..11	k = 2..7
Threshold-row bounds	k = 3..12	k = 3..8

The comparisons use Python integers. Machin's identity π = 16 arctan(1/5) − 4 arctan(1/239) gives a rational enclosure, with truncation errors propagated through every term. The scale is 10^90, and the total interval width is checked to be below 10^−80. The enclosure must decide each π-dependent comparison. If it cannot, the program fails instead of guessing. Agreement with a known 50-digit prefix is only a diagnostic check, not the justification of the enclosure.

For every threshold candidate, the program checks both 2^k C(2m_k,m_k) < 4^(m_k) and failure of that strict inequality at m_k − 1. Above the scan range, the candidate is suggested by the ceiling expression and is accepted only after these exact checks. The global upper error bound is checked without logarithms as 7×4^(D(p)−2) ≤ p. Logarithms use Decimal only to display approximate extrema and do not decide whether a check passes.

In the default range, D(p) changes at 38, 159, 646, 2601 and 10422. Its error D(p) − h(p) is largest at p = 7, about 0.270574, and smallest at p = 10421, about −0.999351. The recorded result states the actual run scope and check count. A normal run returns exit code 0 only after all checks pass. Runtime depends on the machine, especially for the large exact binomial coefficients at k = 12.

These are finite computational checks. The recurrence, math.comb and the addition-only comparison use the same Python integer implementation, so they are not independent arithmetic backends. The general statements rely on the proofs, and the all-k ceiling formula remains open.

6. Open question

Whether p_k = 2⌈4^k/π − 1/4⌉ − k − 1 holds for every k is NOT PROVED. It would follow if the fractional part of 4^k/π − 1/4 never came within about 4^(−k) of an integer. I know of no result on π that settles this. Theorems B1 to B4 and C do not depend on it. Sharper bounds from further terms of the Stirling series would narrow the exceptional band but would not remove it.

7. Remarks

1. The same carry-free principle is used in the work of Islam et al. (2022) [4]. For a given row n, they choose the base b = 10^d with d the number of decimal digits of c(n), so that b > c(n), and read the row off the blocks of (b + 1)^n. The condition c(n) < b is the same one used here. Their paper fixes the row and enlarges the base. This note fixes the base M_p and asks for the largest row. Jiu and Vignat [5] study related digital binomial identities using b-ary binomial coefficients. Those are different objects from the ordinary coefficients used here, and no theorem from that paper is needed in these proofs. Its 2019 correction is linked with the reference.
2. The ingredients are Pascal's rule, Legendre's formula, Robbins' bounds and elementary inequalities. Lemma 2 includes the short modular argument for c(2^M − 1) ≡ 3 (mod 8). This is not a claim of priority.

3. The approximation $N(p) \approx p + \tfrac{1}{2}\log_2(\pi p/2)$ cannot be sharpened to $p + \tfrac{1}{2}\log_2(\pi p/2) + \text{constant} + o(1)$: $N(p) - p$ is an integer, and $N(p) - p - h(p)$ sweeps from about 0 down to almost -1 on each plateau.
4. The earlier note [1] states that from $p = 38$ on, row $p + 3$ is clean as well. Theorem B1 extends this statement to every k.

References

1. J. Augustus, Powers of two as carry-free Pascal rows in Mersenne bases, the preceding note in this repository.
2. H. Robbins, A remark on Stirling's formula, Amer. Math. Monthly 62 (1955), no. 1, 26-29. https://doi.org/10.2307/2308012
3. K. Conrad, Proofs of integrality of binomial coefficients, Section 5, for Legendre's factorial valuation formula.
4. M. S. Islam, M. R. Islam, M. S. Hossan, M. H. Kibria, Generating binomial coefficients in a row of Pascal's triangle from extensions of powers of eleven, Heliyon 8 (2022), no. 11, e11651. https://doi.org/10.1016/j.heliyon.2022.e11651, preprint https://arxiv.org/abs/2005.14490
5. L. Jiu, C. Vignat, On binomial identities in arbitrary bases, J. Integer Sequences 19 (2016), Article 16.5.5. Paper, 2019 correction.
6. OEIS A001405, the central coefficients $C(n,\lfloor n/2 \rfloor)$, as a sequence reference.

Text CC BY 4.0, code MIT.

III. Software and validation scope

`mersenne_rows.py` and `mersenne_rows_gmp.c` generate the selected row n = p + 2. `tests/verify_np.py` performs separate finite checks of the N(p) results. The recorded full run contains 260,608 checks. This historical full run is not presented as having been rerun after every document edit. Quick mode covers p ≤ 1000. Full mode uses p ≤ 20000 and threshold k ≤ 12.

```
python3 -B tests/verify_np.py --quick
python3 -B -m unittest discover -s tests -p 'test_*.py'
```

Finite tests do not replace mathematical proof. The `full` option constructs the complete value and its memory cost grows with the input. Computing one coefficient, generating a complete row and reconstructing the entire number are separate tasks. Earlier PCCPU examples may remain in the archive. The architectural development is described in the CTCPU-Hunter study.

Source repository and related architecture.